StudyProof - Project Overview (Updated)

Version: 1.2.0 | Revised from: original 3-page draft | Date: October 2026

1. What the App Aims to Do

StudyProof is a local-first, cross-platform desktop study-tracking application with a gamified motivation layer and an integrated AI Study Coach. Its goals:

- Make consistent studying visible and rewarding through XP, daily medals, streaks, a calendar, and statistics.
- Keep all user data on-device (atomic JSON in the OS app-data folder, e.g. %APPDATA%\StudyProof), with corrupt-file recovery, schema-version protection, and close-flush of unsaved changes.
- Turn the user's own notes and PDF library into grounded, verbatim-cited AI quizzes and coaching sessions - without sending more context to the model than necessary.
- Ship as a standalone Windows executable (PyInstaller spec + PowerShell build script).

2. Toolkit & Dependencies

Layer	Technology
Language	Python 3.12+
GUI	CustomTkinter (themed dark/light Tkinter wrapper)
PDF processing	PyMuPDF (page-text extraction, rendering)
Imaging	Pillow (medal images, page previews)
AI networking	Standard-library urllib only - no requests/openai SDK; cancellable HTT
Packaging	PyInstaller (StudyProof.spec) + run_build.ps1
Tests	unittest suite under tests/

3. Repository Structure

```
workspace/
··· main.py # Entry point: builds AppState coordinator + GUI
··· StudyProof.spec / run_build.ps1 # PyInstaller packaging (Windows exe)
··· requirements.txt # customtkinter, PyMuPDF, Pillow
··· README.md # Feature documentation (v1.2.0)
··· docs-style plans # new_requirements_document.md,
· # "AI Requirements Document - StudyProof v2.md",
· # AI_implementation_plan.md,
· # study_coach_optimisation_plan.md,
· # AUTO_SCROLL_PLAN.md
··· assets/ # Medal artwork, icons
··· scripts/ # Helper/build scripts
··· tests/ # Unit tests (domain rules, persistence, coach logic)
··· app/
··· constants.py # Thresholds: XP blocks, medal minutes, limits
··· core/ # PURE domain logic - no GUI, no I/O
··· state/ # AppState coordinator (GUI · core bridge)
··· persistence/ # Atomic JSON save/load, backup + quarantine
··· gui/ # CustomTkinter screens, widgets, event queue
··· utils/ # Shared helpers
```

Architecture rule: strict layering. core contains business rules only; persistence owns disk format and atomic writes; gui never touches files directly - it communicates through AppState and a thread-safe event queue so background work (PDF extraction, AI calls) never blocks the UI.

4. Core Domain Rules (app/core)

- XP engine: 10 XP per completed 10-minute block; partial blocks earn nothing. Sessions crossing midnight are attributed to their start day.
- Daily medals: Bronze >= 120 min, Silver >= 180 min, Gold >= 300 min per day (image when available, text fallback).
- Streaks: a qualifying day needs >= 10 studied minutes; today does not break the streak until the day ends. Current & longest streaks shown on Stats.
- Session logging: one-tap presets (30 min / 1 h / 2 h) plus a custom dialog validated to 5 min · 12 h.
- Stopwatch: monotonic-clock timing; resetting a run of >= 10 minutes auto-logs it as a custom session rounded down to the nearest 10 minutes.
- Notes: titled free-text notes (120-char titles, 100k bodies), optionally attached to a calendar day; searchable; autosaved immediately; invalid records skipped on load.
- Calendar: Monday-first month grid, medal-colored cells, day-details dialog with sessions + notes.
- Stats: weekly/monthly/lifetime totals, active days, averages, medal counts, total-notes count.

5. Persistence (app/persistence)

- Single JSON save file (studyproof_save.json) holding sessions, notes, settings (theme), and coach history.
- Atomic writes (write temp · replace) so a crash cannot half-corrupt data.
- On corruption: automatic backup + quarantine recovery path; schema version guard rejects newer-format files rather than damaging them.
- Manual "Save" and safe "Load from file" flow: validate · confirm · backup · replace.

6. AI Integration - The Study Coach

The most distinctive part of the app. Design principles (from the v2 requirements):

1. Bring-your-own-endpoint: talks to any OpenAI-compatible chat API via urllib with a user-supplied base URL + API key. Calls run on a worker thread and are cancellable.
2. Grounded in the user's material: questions and feedback are generated from the user's notes/PDF text using deterministic context packing - a fixed budget of source snippets is assembled identically for the same inputs, keeping behavior reproducible and costs bounded.
3. Verbatim citation grounding: every coach claim must quote source text verbatim; ungrounded output is rejected/retried - this is the anti-hallucination mechanism.
4. Dual-track design: a visible Coach converses with the student while a hidden Assessor scores each answer, identifying weak topics that feed the "Revise Weak Topics" feature and Statistics.
5. Integrity rules (A-25): the learner's first answer is immutable once submitted; confidence is captured exactly once - after the first answer, before any hint/feedback is revealed; hints/feedback are gated on both existing; revisions append only. The coaching log is an integrity-protected record, not an editable draft.
6. Accessibility: role prefixes ("Coach:" / "You:"), textual verified ticks, tooltips explaining gated/disabled buttons (ACC-3/4/5).
7. Persistence: conversations are saved in the local JSON store; no study content leaves the machine except to the user-configured endpoint.

7. Notable Updates vs. the Original 3-Page Draft

- Version bump: the app now documents itself at v1.2.0; the draft predates several shipped features.
- Notes system (list + detail pane, day attachment, search, validation limits) is now fully implemented and persisted alongside sessions - the draft treated notes as planned-only.
- Study Stopwatch with auto-logging rules (>= 10 min · rounded-down session) is a new sidebar feature absent from the draft.

- Calendar UX refinements: today's tile now shares the header "Today" button's blue accent so the two always match; light/dark grey tiles specified.

- Coach maturity: Statistics now lists the same specific weak topics as Revise Weak Topics and opens revision sessions directly - without exposing scores or confidence ratings to the user.
Immutable-first-answer, hint gating, and accessibility items (role prefixes, textual ticks, tooltips) are implemented per the v2 spec.

- PDF Library hardening: background text extraction, zoom + text view, and friendly guidance for protected / oversized / textless files; imported files are copied into the local library.

- Resilience documented: corrupt-file backup + quarantine, schema-version protection, and close-flush are now explicit guarantees, matching the shipped persistence code.

- Build pipeline: PyInstaller spec + PowerShell build script produce the standalone Windows executable referenced throughout.

8. How to Run / Build

```
pip install -r requirements.txt
python main.py # run the app
pyinstaller StudyProof.spec # package (see run_build.ps1 on Windows)
python -m unittest discover tests # run the test suite
```